Accuracy and Order Sensitivity Diverge Under Label-Free Strategies

Karl Hanna
Queen's University Belfast
khanna28@qub.ac.uk

Chen Feng
Queen's University Belfast
c.feng@qub.ac.uk

Abstract

Multiple-choice benchmarks are widely used to evaluate large language models, but MCQ scores conflate knowledge with sensitivity to option order, which makes them unreliable measures of model knowledge. In this paper, we test whether preventing a model from seeing option labels while committing to an answer removes positional influence and, in turn, improves performance. We evaluate two different strategies for mitigating bias. The first uses a generation-then-matching approach, and the second scores options in isolation, which is positionally unbiased by construction. Neither reliably improves accuracy. A complete decomposition shows that the bottleneck is withholding options, not the matching step. The only configuration that consistently matches the baseline is the one that shows the model all options paired with an LLM matcher. However, eliminating positional influence entirely still does not reliably yield accuracy gains, while cyclic permutation often improves them. For two-stage prompting, an aggregate measure of recall imbalance and a direct per-question measure of order sensitivity both fail to show reliable debiasing.

1 Introduction

Multiple-choice question (MCQ) benchmarks are one of the dominant evaluation formats (Hendrycks et al., 2021; Clark et al., 2018), since they are inexpensive to run compared to other formats and can be scored automatically. However, the score is only as trustworthy as the evaluation is robust. A model's accuracy on an MCQ benchmark does not distinguish knowledge from sensitivity to option order (Pezeshkpour and Hruschka, 2024; Zheng et al., 2024; Wang et al., 2025). This has motivated a range of methods for reducing option-order effects.

A lot of these mitigation strategies impose either a cost or an access requirement. Assuming that k denotes the number of options per question, cyclic permutation and full permutation need no access to logits, but they cost k and $k!$ calls per question, which is not always feasible. PriDe (Zheng et al., 2024) costs one call per question with an additional fixed calibration overhead, but it requires logit access that many providers do not expose. These constraints motivate approaches that obtain robustness from prompt design rather than from repeated querying or logit access.

In this paper, we test the premise that if a model never sees option labels while committing to an answer, option order cannot influence the prediction. The first strategy is two-stage prompting. The model first receives the question alone and generates a free-text answer. Then, in Stage 2, the model matches that free-text answer to the option that most closely corresponds to it. The second strategy, independent hypothesis scoring, takes a different approach. The model is prompted k times; each call presents the question with a single option and asks the model to score it. After all options are scored, the highest-scoring option is selected. This strategy is positionally unbiased by construction, since each call is isolated and there are no positions to influence the response. We test six models spanning proprietary and open-weight systems across two benchmarks.

We find that neither strategy reliably improves accuracy. Two-stage prompting reduces it for nearly all model–benchmark pairs, while independent hypothesis scoring is more mixed: it reduces accuracy for most models, but improves Llama-local, with the gain concentrated on ARC-Challenge. We decompose two-stage prompting along two factors: whether options are hidden or visible in Stage 1, and whether Stage 2 uses an LLM call or embedding-based semantic matching. The decomposition shows that when the Stage 2 LLM call is replaced by semantic matching with options hidden, performance drops sharply because

the matcher cannot reliably map the free-text answer to an option. However, when paired with a Stage 1 where the options are visible, much of the loss is recovered, which indicates that the bottleneck is hiding the options in Stage 1 rather than the matching step.

To measure positional effects, we use flip rate as a direct per-question measure of order sensitivity and recall standard deviation (RStd) to measure how unevenly the model performs across answer positions. Two-stage prompting does not reliably improve either measure: some model–benchmark pairs become more stable or balanced, while others become less so. Moreover, lower positional sensitivity does not necessarily coincide with higher accuracy; for GPT-4.1 mini on MMLU, flip rate roughly halves while accuracy falls. Independent hypothesis removes positional influence by construction, yet accuracy still does not consistently improve, and only Llama-local sees a meaningful increase. Taken together, reducing positional effects, even eliminating them entirely, does not always translate into accuracy gains, which is consistent with prior observations that removing option identifiers reduces selection bias while degrading accuracy (Zheng et al., 2024).

Our contributions are as follows. We evaluate two multiple-choice prompting strategies against established baselines across six models and two benchmarks; we carry out a complete 2×2 decomposition isolating where two-stage prompting fails; we compute flip rate as a direct per-question measure of option-order sensitivity and RStd to measure how unevenly the model performs across answer positions; and we show that reducing positional effects and improving accuracy come apart across the strategies evaluated, including one that removes positional influence by construction and still does not consistently improve accuracy.

2 Related Work

Multiple-choice brittleness and option-order sensitivity. Several papers show that large language models are not robust multiple-choice reasoners, even when their benchmark accuracy looks strong. Zheng et al. (2024) find that LLMs behave as unreliable multiple-choice selectors, which motivates methods that reduce sensitivity in the final selection step. Pezeshkpour and Hruschka (2024) show that performance changes substantially when the options are reordered, which suggests that multiple-choice evaluation may be measuring sensitivity to formatting rather than stable underlying knowledge. Wang et al. (2025) raise a related concern, arguing that models may succeed at MCQA by selecting the least incorrect option rather than identifying a clearly justified correct one. More recent work continues this line, showing that benchmark performance can also be distorted by hidden or disguised biases in option-based evaluation (Nowak et al., 2026).

Debiasing and robustness-oriented prompting methods. Several approaches try to make multiple-choice evaluation more robust by modifying the prompting or selection procedure. PriDe (Zheng et al., 2024) is a central example, estimating a prior from a small calibration set and subtracting it from the prediction distribution. BiasPrompting (Vu et al., 2025) also treats bias as a central issue in MCQ answering and proposes a prompting-based intervention aimed at improving behaviour under bias-sensitive conditions. Both treat robustness as a core requirement for reliable evaluation rather than a minor implementation detail. Our work is closest to this line, but differs in evaluating two label-free strategies and using diagnostic variants to isolate which stage is responsible for the observed failures.

Open-style answering and answer matching. Another line of work questions whether forced-choice evaluation is the right interface for measuring model knowledge at all. Myrzakhan et al. (2024) argue for moving from purely multiple-choice formats toward open-style questions in leaderboard-style evaluation, since multiple-choice constraints can distort what is actually being measured. Chandak et al. (2025) go further and show that answer matching outperforms standard multiple-choice evaluation, which suggests that free-form responses give a more faithful picture of model competence than letter selection alone. This directly motivates two-stage methods: if the model first answers in open form and is only later mapped back to the provided options, we can test whether part of the observed MCQ brittleness comes from the selection interface rather than from the model's ability to solve the problem.

LLM-as-judge and positional bias. The matching step in two-stage prompting is structurally an instance of LLM-as-judge evaluation, since the model is asked to compare its own free-text re-

sponse against a set of labelled options and select the closest match. This raises a concern that is well documented in the judge reliability literature. Zheng et al. (2023) show that LLM judges exhibit position bias, verbosity bias, and self-enhancement bias, with GPT-4 producing order-inconsistent verdicts on a substantial fraction of cases where response quality is similar. If the matching step inherits the same positional sensitivity as direct MCQ selection, then two-stage prompting may relocate the bias rather than remove it. Bias relocation is therefore plausible, but our results indicate that the effect is model-dependent.

Isolated per-option scoring. A separate line of work removes option-order effects structurally rather than correcting them after they occur. Set-Based Prompting (McIlroy-Young et al., 2024) modifies the attention mask and positional encoding so that option order cannot affect the output. Its effect on accuracy is small, typically remaining within the variation caused by reordering under ordinary prompting: order invariance is achieved, but accuracy is essentially unchanged. Balepur et al. (2024) evaluate each option separately under both question-present and choices-only settings. Their question-present setup closely resembles independent hypothesis scoring, but elicits a binary correctness judgement rather than a numeric score. In the choices-only setting, they find that judging options independently underperforms presenting them together, suggesting that models benefit from comparisons among the options. A similarly related setup is the multiple-choice verification condition tested by Chandak et al. (2025), where the model receives the question with each choice separately and independently judges whether that choice is correct. Their method counts a question as correct only when the gold choice is marked true and every distractor false, whereas we elicit a numeric score for each option and select the highest. They find that verification produces an accuracy estimate close to answer matching but aligns substantially worse with ground-truth evaluation, and they argue formally that verification is strictly harder than discrimination. Independent hypothesis scoring likewise eliminates positional effects through prompt-level isolation, requiring no architectural modification and therefore remaining applicable to closed APIs. Together, these findings suggest a potential cost to option isolation: it prevents the model from directly comparing the available choices.

Positioning of this work. This paper sits at the intersection of these strands. Prior work has shown that MCQ evaluation is vulnerable to option-order effects, selector bias, and other formatting-sensitive artifacts (Zheng et al., 2024; Pezeshkpour and Hruschka, 2024; Wang et al., 2025; Nowak et al., 2026), while separate work has argued that open-form answering or answer matching may provide a better measurement interface (Myrzakhan et al., 2024; Chandak et al., 2025). These strands are largely developed in isolation, and cases where bias reduction fails to improve accuracy are reported incidentally rather than examined directly. We evaluate two label-free strategies against established baselines across six models and two benchmarks, decompose two-stage prompting into a 2×2 grid to isolate which factor drives its failure, and measure order sensitivity per question under permutation.

3 Methodology

3.1 Benchmarks and Sampling

MMLU (Hendrycks et al., 2021) is a benchmark dataset containing 14,042 questions in English across 57 subjects, covering a broad range of fields and difficulty levels. In this paper, we use 20 questions from each of 50 subjects, sampled with seed 42, with the following seven subjects excluded: `human_aging`, `human_sexuality`, `management`, `marketing`, `miscellaneous`, `moral_disputes`, and `public_relations`. ARC-Challenge (Clark et al., 2018) is the harder subset of the ARC dataset, consisting of questions in English that retrieval-based methods failed to answer correctly. It contains 1,172 grade-school level science questions, of which 1,000 random ones were used, sampled using seed 42. For PriDe, an additional 50-question calibration set was sampled from the benchmark pool using seed 42 and excluded from the 1,000-question evaluation split by construction. MMLU is released under the MIT license and ARC-Challenge under CC-BY-SA; both are used here for their intended research purpose.

3.2 Models

The six models evaluated are GPT-4.1 mini (OpenAI et al., 2024), Gemini 2.5 Flash (Comanici et al., 2025), Llama 3.1 8B Instant (Groq API) (Grattafiori et al., 2024), Qwen 2.5 7B Instruct Turbo (Together AI API) (Qwen et al., 2025), Qwen 2.5 7B Instruct (local), and Llama 3.1 8B Instruct (local). The API models span proprietary

and open-weight systems across multiple providers and were accessed under their respective terms of service. The local Qwen2.5 and Llama 3.1 models were used under the Apache License 2.0 and Llama 3.1 Community License, respectively. They were run via Hugging Face Transformers on SLURM, using a 3g.40gb MIG slice depending on cluster availability, enabling comparison with their API-served counterparts. PriDe is evaluated on Qwen 2.5 7B Instruct Turbo, Qwen 2.5 7B Instruct (local), and Llama 3.1 8B Instruct (local), as these models fully expose log-probabilities. A temperature of 0.0, `max_tokens` of 500, and seed 42 were used across all models; the independent hypothesis strategy overrides `max_tokens` to 4000. Per-model concurrency limits and rate constraints are listed in Appendix C.

3.3 Problem Formulation

Let M denote a language model, Q a multiple-choice question, and $O = \{o_i\}_{i=1}^{N}$ its set of semantic answer options. Let ϕ denote the presentation of these options, including their ordering and assignment to labels such as A, B, C, and D. We use $A \in O$ to denote the model's semantic prediction after mapping its emitted label back to the corresponding option text.

Ideally, the prediction would depend only on the question and the option set. In practice, however, the model receives a particular labelled and ordered presentation, so its prediction is modeled as

$$P_M(A \mid Q, O, \phi).$$

The language wrapper surrounding the question is fixed across conditions and is therefore omitted from the notation. Positional sensitivity occurs when changing only ϕ changes the model's semantic prediction for the same Q and O.

3.4 Strategies

The two strategies differ in whether or not the final prediction remains dependent on the joint option presentation ϕ. **Two-Stage Prompting** first generates a free-text response E without access to the answer options:

$$E = M_{\text{gen}}(Q)$$

denotes the model under the Stage 1 prompt. Then, in Stage 2, the model's prediction is modelled as

$$P_M(A \mid Q, E, O, \phi).$$

We can see that ϕ is removed from Stage 1, where the model produces evidence for Stage 2, but is reintroduced in Stage 2, meaning that the strategy is not positionally invariant by construction.

Independent Hypothesis prompts the model N times and asks it to score each presented option using

$$s_i = M_{\text{score}}(Q, o_i).$$

where s_i is between 0 and 100. The final answer is then chosen using

$$\hat{A} = o_{\arg\max_i s_i}.$$

If multiple options share the highest score, a seeded pseudo-random tie-break selects among them. Since each score depends only on the question and one option, rather than on a jointly ordered and labelled option list, the final prediction does not depend on ϕ. Evaluating options independently can make some questions harder, since questions phrased as "which of the following" may be underspecified when the remaining options are hidden.

The full prompt templates for each condition are provided in Appendix A.

3.5 Baselines

Three conditions were used for comparison. The **baseline** presents the question directly in multiple-choice format. **Cyclic permutation** queries the model k times with options rotated across all positions; responses are unpermuted and a majority vote determines the final answer, with ties broken by defaulting to the original permutation. **PriDe** (Zheng et al., 2024) uses a calibration set to estimate a positional prior, which is used to adjust logit-based prediction probabilities; it is evaluated only on models which provide true log-probabilities.

3.6 Diagnostic Grid

We use a 2×2 diagnostic grid to isolate different factors of two-stage prompting. Stage 1 is presenting the options in the prompt or withholding them, and Stage 2 is using an LLM to match or using embedding matching. Hidden options + LLM matching is two-stage, hidden + embedding is semantic matching, visible + embedding is `text_extraction`, and visible + LLM matching is the final cell. The fourth cell reuses the `text_extraction` Stage 1 outputs, so the only difference is the matching stage. all-MiniLM-L6-v2 is the model used for semantic matching. Matching

proceeds as a cascade: an exact match is taken first, then substring containment, which is only considered when the contained substring is a minimum of length 4 and is unique, and finally cosine similarity, where only matches above 0.30 are accepted, which is why an argmax over cosine can still be unscorable. Ties at the cosine-similarity stage, and collisions arising from string normalization at the exact-match stage, are resolved by a seeded random draw over the tied canonical option content, keyed on the run seed and question ID, so that resolution does not depend on display position.

3.7 Prompt Ablations

Two prompt ablations were evaluated to test whether the primary results are sensitive to the specific prompt wording used. The Stage 1 ablation (v2) modifies the free-text prompt used in two-stage prompting while keeping the Stage 2 matching prompt unchanged. The Stage 2 ablation (v3) modifies the matching prompt while keeping the Stage 1 prompt unchanged. Both ablations are evaluated on all six models across both benchmarks under the LLM-based matching condition. The original two-stage prompt is referred to as v1 throughout.

3.8 Metrics

Two accuracy metrics are reported. End-to-end accuracy is defined as the number of correct answers divided by the total number of questions; unscorable outputs are counted as incorrect. Conditional accuracy is defined as the number of correct answers divided by the number of scorable outputs only. An output is considered unscorable if the final response cannot be parsed as A, B, C, or D, or if an API error occurs. For two-stage prompting, parse failures occur specifically at the option-matching stage. We also report recall standard deviation (RStd), following Zheng et al. (2024). For each method and model, we compute recall separately for questions whose gold answer appears in positions A, B, C, or D, then take the population standard deviation of the four resulting recalls and report it in percentage points. Lower values indicate more uniform recall across answer positions. Unlike flip rate, RStd compares different subsets of questions rather than counterfactual orderings of the same questions, so we treat it as an aggregate diagnostic rather than a direct measure of order sensitivity. We measure flip rate across all six models by cyclically permuting the answer options and recording whether the model's semantic answer changes. Each question receives four permutations, except the three ARC-Challenge questions that have only three options, which receive three. After mapping each parsed answer label back to its canonical option content, a question is counted as flipped if it produces at least two distinct semantic answers across its permutations. For two-stage prompting, the Stage 1 outputs are reused so that only the matching stage is re-run.

3.9 Statistics

For accuracy, we use Clopper–Pearson intervals. For RStd, we use 10,000 question-level bootstrap resamples at seed 42, resampling from the complete set of question outputs and recomputing the scored subset, the four per-position recalls, and their population standard deviation within each resample; we report 95% percentile intervals. We use McNemar's test (asymptotic) to compare each method against that model's own baseline, restricted to questions scored under both conditions. For the tie-breaks in independent hypothesis, we use 10 alternate seeds, excluding the original from the summary statistics.

3.10 Code Availability

All experiment code, prompt templates, and evaluation scripts, including the fourth diagnostic-cell runner and flip-rate trace pipeline, are publicly available under the MIT license at https://github.com/cotenthusiast/choicebench

4 Results

4.1 Accuracy

The two-stage strategy reduces end-to-end accuracy in 11 of 12 model–benchmark pairs, while independent hypothesis reduces it in 8 of 11, with Gemini on ARC-Challenge excluded because the run suffered extensive provider-side API failures and was never completed. For two-stage, Gemini drops from 84.9 to 68.3 on MMLU and from 96.9 to 86.6 on ARC, both largely due to parse failures. Llama-local on ARC is the only pair that rises, albeit only from 58.0 to 58.3, which is well within noise. GPT-4.1 mini produced zero parse failures yet still fell from 81.8 to 80.1, which shows that the loss in accuracy is not only because of the parsing.

Independent hypothesis produces mostly negative results, although smaller in magnitude than two-stage. Qwen-API drops on MMLU from 68.9

to 66.3, Llama-API on MMLU from 65.2 to 60.5, and Gemini on MMLU from 84.9 to 83.4. The exceptions are GPT-4.1 mini on MMLU, going from 81.8 to 83.0, which is within noise, and Llama-local, which rises on both benchmarks. We return to this model in Section 4.7 below, where other methods recover comparable gains.

Cyclic permutation improves 5 of 6 pairs on MMLU and 5 of 6 on ARC. Table 2 reports the full accuracy results with confidence intervals; the remaining results tables for both benchmarks are given in Appendix I.

4.2 End-to-end vs conditional accuracy

Under two-stage prompting, Gemini's conditional accuracy is 82.0 while its end-to-end accuracy is only 68.3, a gap of 13.7 points caused entirely by parse failures at the matching step. This shows why conditional accuracy alone is insufficient when evaluating debiasing methods. Table 4 reports the number of scored questions for every method that produces unscorable outputs. In contrast, independent hypothesis scores a full 1,000/1,000 in every included cell, while cyclic is near complete, so it is reasonable to conclude that the distinction lies in the generation-then-matching idea.

4.3 Recall standard deviation

RStd is reported for all six models across both benchmarks in Table 6 (MMLU) and the corresponding ARC-Challenge table in Appendix I, following Zheng et al. (2024). Two-stage prompting increases RStd relative to baseline in 5 of the 12 model–benchmark pairs and decreases it in the remaining 7, so it does not reliably reduce recall imbalance across answer positions. Llama-local (local) has by far the highest baseline RStd on both benchmarks (12.5 on ARC, 10.2 on MMLU), consistent with its extreme flip rates, and two-stage reduces it on both (to 8.0 and 8.1 respectively), tracking the same direction as its flip-rate reduction.

The two metrics broadly agree in direction: Gemini, Llama-API, and Qwen-API (Turbo, on ARC) all show flip rate and RStd increasing together. The clearest exception is Qwen-local, where the two metrics move in opposite directions on both benchmarks: flip rate rises under two-stage (+5.2 pp on MMLU, +12.2 pp on ARC) while RStd falls (8.06 to 4.27 on MMLU, 4.68 to 3.05 on ARC). This shows two-stage can narrow a model's aggregate letter-level imbalance while simultaneously making its individual answers more sensitive to option order, so the two metrics capture related but distinct failure modes and are not interchangeable.

Semantic matching yields the lowest RStd values for nearly every model (e.g. 1.0 on ARC for Gemini), consistent with its near-zero flip rate, though its scored subset is substantially smaller for several models (as low as 688 of 1,000 questions for Llama-local on MMLU), so part of this reduction may reflect the smaller, potentially non-representative sample rather than bias elimination alone. Under independent hypothesis, RStd is small but nonzero for every one of the eleven valid model–benchmark cells (1.1 to 3.9; full values in Table 13). Because this strategy contains no display positions, these values show that observed RStd can remain nonzero because of finite-sample variation or differences in difficulty across gold-position subsets; they should not be interpreted as residual positional bias.

4.4 Flip Rates

To measure option-order sensitivity directly, we compute flip rates for all six models across both MMLU and ARC-Challenge (Table 8). Two-stage prompting increases the flip rate relative to baseline in 7 of the 12 model–benchmark pairs and decreases it in the remaining 5, so, as with RStd (Section 4.3), it does not reliably reduce order sensitivity, and more often increases it than not. Llama-local (local) again stands out: its baseline flip rate is by far the highest of any model (74.6 on ARC, 80.3 on MMLU), and two-stage reduces it substantially on both (to 55.8 and 63.1 respectively), the only model for which the reduction is this large. GPT-4.1 mini's flip rate falls by roughly half on MMLU (21.6 to 11.8) while its accuracy also falls (81.8 to 80.1), consistent with the decoupling thesis: reduced order sensitivity does not imply improved accuracy.

4.5 Broken/Fixed

McNemar's test indicates whether the broken-versus-fixed imbalance is larger than expected by chance. On MMLU, only Gemini's damage is distinguishable, with 87 broken and 23 fixed ($p < .001$), while the others are all indistinguishable from chance. Llama-local's run had 205 questions that were broken and 218 were fixed among co-scored items: a net gain of 13. On ARC it is different: the damage is real for five of six models, with only Llama-local indistinguishable at 174

broken and 210 fixed ($p = 0.074$).

Cyclic provides the contrast, with far less churn. For all semantic matching cells, the p-value was $< .001$, except for Llama-local on MMLU at $p = 0.003$. Table 9 reports the broken and fixed counts for every method.

4.6 The 2×2 grid

Semantic matching paired with hidden options particularly stands out because of the sharp drop. However, that same drop is not present when semantic matching is paired with visible options; we can even see that Gemini on MMLU under the visible options with semantic matching beats the LLM matcher. Therefore we can conclude that the matcher is not the problem, but that hiding the options makes the matching more difficult.

Under semantic matching Llama-local scores 32.0 and Gemini 48.8 on MMLU, while parse failures range from 14.6% on Qwen-API to 31.2% on Llama-local. Showing options recovers much of the loss, most notably GPT-4.1 mini on MMLU rising from 45.8 to 81.7.

When comparing Visible+LLM matching to baseline, the majority of the differences are within noise, except for Llama-local increasing by 6.5 pp on MMLU and 12.4 pp on ARC, both having non-overlapping CIs and McNemar $p < .001$. Table 11 reports all four cells of the grid alongside the baseline.

4.7 Independent Hypothesis

Since ties are broken randomly, the reported accuracy is one draw, and the gain only counts if it survives other seeds. On ARC it does: 72.4 reported, 72.5 mean across ten seeds, ranging from 71.6–73.2. On MMLU the relationship is reversed: the reported score (seed 42) is 51.2, which falls below the entire ten-seed range rather than sitting at its low end, the mean across the ten new seeds is 52.6, ranging from 51.6–54.0. Either way, the defensible gain on MMLU is modest, roughly 1.0–2.4 pp, and does not support treating this as a reliable effect.

Looking at Llama-local, we see an increase of 14.4 pp with independent hypothesis on ARC, which looks like proof that eliminating positional bias buys accuracy. However, looking at unrelated methods suggests otherwise. On ARC, Llama-local achieves a baseline score of 58.0, then increases to 72.9 under cyclic, 72.4 under independent hypothesis, and 70.4 under Visible+LLM. On MMLU, the baseline score is 50.2, and cyclic is 57.0, Visible+LLM is 56.7, and independent hypothesis is 51.2. On ARC, three unrelated methods converge to a similar gain, which tracks Llama-local's unusually high recoverable performance rather than anything specific to removing positional information. On MMLU, however, independent hypothesis's gain is markedly smaller and within noise, while cyclic and Visible+LLM deliver larger, more defensible gains, suggesting the benchmark, not just the model, modulates how much performance these methods can recover.

Table 13 reports the accuracy, tie rates, and seed spread for each cell.

4.8 Fallback

Substituting the baseline prediction for unscorable outputs separates losses caused by parse failures from genuine matching errors. Under two-stage on MMLU, the only meaningful gains are 2.3 pp on Llama-local and 11.3 pp on Gemini. Only Gemini's loss was parse-driven, and even at 79.6 it stays below its 84.9 baseline. Table 14 reports the fallback results for every affected method.

4.9 Ablations

Alongside the original prompt, we evaluate two variations of the setup, one changing the Stage 1 prompt and one changing the Stage 2 prompt. The pattern holds and the negative results persist under both. Gemini, the most affected model, scores 70.2 and 70.7 under the two variants, against 68.3 under the original prompt and a baseline of 84.9. All prompts used are given in Appendix A.

5 Discussion

For two-stage, we can predict that the loss in performance stems from Stage 1 not seeing the options, so questions that rely on the choices to convey the intended answer have no answerable free-text form. To support this, we can see that there is a gap of 35.9 pp for GPT-4.1 mini on MMLU when using semantic matching with options hidden versus options visible. Additionally, Chandak et al. (2025) report that when filtering questions in MMLU-Pro and GPQA-Diamond to uniquely answerable ones, the questions are cut by more than half; our sets are unfiltered.

For independent hypothesis, one reason for the poor performance may be that each option is scored alone, and there is loss of comparative context

which may help with answering the questions. Balepur et al. (2024) classify each choice independently and find that priors over individual choices do not fully explain choices-only accuracy, which implies that models exploit group dynamics among options. Chandak et al. (2025) test an approach which is extremely similar to independent hypothesis. They present the model with each choice separately and ask the model to judge it independently although their approach uses boolean judging while we use variable scoring. They also argue that verification is strictly harder than discrimination. A tie happens in our strategy when two or more of the options share the same score, so the model expresses no preference and the outcome is decided by a seeded random draw over the tied option content. This rate ranges from 6.2% on GPT/ARC all the way up to 43.2% on Llama-local/MMLU, which signals model weakness.

The two label-free strategies do not hold up as well as cyclic permutation; as shown for GPT-4.1 mini on MMLU, reduced order sensitivity under two-stage does not translate into higher accuracy. This is further supported by Zheng et al. (2024), who find that removing option identifiers reduces selection bias while usually degrading accuracy.

Comparing to Chandak et al. (2025) specifically, our methods differ in several respects. Firstly, their matcher compares the response against the reference answer, while our second stage is presented with the response and all four options and is asked to choose the one most closely matching it. Additionally, they measure alignment with the ground truth (Scott's π), and not accuracy. MCQ gives the highest accuracy of any grader while aligning the worst, so a lower score is not automatically worse in their framing. These differences mark where the approach holds: matching succeeds when the matcher verifies against a reference answer, and fails when it must select among unlabelled options.

When taking costs into account, baseline requires one call per question, two-stage requires two, while cyclic and independent hypothesis both require k calls. Independent hypothesis requires the same number of calls as cyclic, but loses to it in 10 of 11 pairs. Two-stage prompting is cheaper than cyclic, though it does not deliver comparable results. Cyclic is the best-performing model-agnostic method among those evaluated. PriDe can substantially reduce accuracy even when log-probabilities are available, as shown by Llama-local dropping from 50.2 to 44.2 on MMLU and from 58.0 to 48.8 on ARC-Challenge.

6 Conclusion

This paper investigated whether two label-free strategies that separate answer commitment from option presentation can reduce positional effects and improve accuracy. We tested the strategies across six models and two benchmarks and evaluated them against established baselines. Neither strategy reliably improves accuracy: two-stage prompting degrades performance in 11 of 12 model–benchmark pairs, while independent hypothesis degrades it in 8 of 11 valid pairs.

Our 2×2 decomposition pinpoints the bottleneck on withholding the options rather than the matching step. The only configuration that consistently matched baseline was having visible options in Stage 1, paired with an LLM for matching.

For two-stage prompting, neither diagnostic shows reliable improvement: RStd increases in 5 of 12 model–benchmark pairs, while flip rate increases in 7 of 12. The two diagnostics move in the same direction in 8 of the 12 pairs, with Qwen-local the clearest exception: two-stage narrows its aggregate letter-level imbalance on both benchmarks while simultaneously making individual answers more sensitive to option order. Under independent hypothesis, RStd remains small but nonzero across all 11 valid cells; because the strategy contains no display positions, these values illustrate that RStd also reflects finite-sample and between-subset variation rather than positional influence alone.

Independent hypothesis eliminates positional influence by construction, yet accuracy does not reliably improve; where it does, the magnitude of the gain can be sensitive to the random tie-break seed, particularly on MMLU. Semantic matching drives flip rate to exactly zero for every model on both benchmarks, confirming its position-blindness by construction, but at a substantial cost to accuracy. Cyclic permutation improves accuracy in 10 of 12 pairs, although its gains cannot be attributed solely to reduced positional effects because it also aggregates predictions across permutations. Taken together, these results show that eliminating positional influence does not reliably improve accuracy, while two-stage prompting does not reliably eliminate that influence in the first place.

Limitations

We evaluate six models on two benchmarks in a four-option setting with a fixed number of questions per benchmark, so it is unclear whether these results hold for settings with more options, different question distributions, or larger models. Additionally, every condition was run once, with the exception of the tie-break seed sweep for independent hypothesis, so we do not test run-to-run variance from provider-side non-determinism. We also do not include repeated identical-order controls in the flip-rate experiment, so residual provider-side non-determinism may contribute to observed flips, particularly for API models.

Each strategy was only evaluated in a single instantiation. For two-stage, we use one free-text prompt and one matching prompt, along with two ablations that vary each stage independently. However, neither of the ablations enables reasoning in Stage 1, and the v2 ablation strengthens the reasoning suppression already present in v1. This means no condition allows the model to reason before answering, despite this being the variant most directly suggested by the answer-matching literature. We also do not test a separate or stronger matcher model. For PriDe, the calibration set size was not varied, so its behaviour is only evaluated at a single calibration budget.

Several cells, most notably semantic matching across all six models, and Gemini's two-stage cells on both benchmarks, score substantially fewer than 1,000 of 1,000 questions due to parse failures, so their flip rate and RStd values reflect only the scored subset rather than the complete question set, which may inflate position-blindness for affected cells. The under-specification mechanism we propose is also supported by prior work rather than tested on our own data, and stratifying results by question type would evaluate it directly.

Finally, there are two data issues to note. Independent hypothesis on Gemini and ARC-Challenge is excluded because the run suffered extensive provider-side API failures, leaving only 313 of 1,000 questions scored. Three of the 1,000 ARC-Challenge questions also have three options rather than four, and cloud-side prompts rendered a placeholder fourth option for these items. It is also worth mentioning that parse failure rates depend on provider-specific serving behaviour, and Gemini produced substantially more unscorable outputs than the other models.

Ethical Considerations

Generative AI tools were used to assist with both the codebase and manuscript. In the codebase, AI assistance was used for refactoring, debugging, later feature implementation, and project-structure improvements. In the manuscript, AI assistance was used for wording refinement, LaTeX formatting, table presentation, consistency checking, and claim clarification. The authors remain responsible for the experimental design, implementation, analysis, interpretation, and final claims.

This is an evaluation-methodology study using public benchmarks with no human subjects or new data collection, so risks are minimal; the main consideration is that debiasing methods reported as ineffective should not be relied upon as safety guarantees.

References

Nishant Balepur, Abhilasha Ravichander, and Rachel Rudinger. 2024. Artifacts or abduction: How do LLMs answer multiple-choice questions without the question? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10308–10330.

Nikhil Chandak, Shashwat Goel, Ameya Prabhu, Moritz Hardt, and Jonas Geiping. 2025. Answer Matching Outperforms Multiple Choice for Language Model Evaluation. *arXiv preprint arXiv:2507.02856*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think You Have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, Luke Marris, Sam Petulla, Colin Gaffney, Asaf Aharoni, Nathan Lintz, Tiago Cardal Pais, Henrik Jacobsson, Idan Szpektor, Nan-Jiang Jiang, and 3416 others. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *Preprint*, arXiv:2507.06261.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt.

2021. Measuring massive multitask language understanding. In *International Conference on Learning Representations*.

Reid McIlroy-Young, Katrina Brown, Conlan Olson, Linjun Zhang, and Cynthia Dwork. 2024. Order-independence without fine tuning. *Advances in Neural Information Processing Systems*, 37:72818–72839.

Aidar Myrzakhan, Sondos Mahmoud Bsharat, and Zhiqiang Shen. 2024. Open-LLM-Leaderboard: From Multi-Choice to Open-Style Questions for LLMs Evaluation, Benchmark, and Arena. *arXiv preprint arXiv:2406.07545*.

Mateusz Nowak, Xavier Cadet, and Peter Chin. 2026. ABCD: All Biases Come Disguised. *arXiv preprint arXiv:2602.17445*.

OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, and 262 others. 2024. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.

Pouya Pezeshkpour and Estevam Hruschka. 2024. Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2006–2017, Mexico City, Mexico. Association for Computational Linguistics.

Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, and 25 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Duc Anh Vu, Thong Nguyen, Cong-Duy Nguyen, Viet Anh Nguyen, and Anh Tuan Luu. 2025. More Bias, Less Bias: BiasPrompting for Enhanced Multiple-Choice Question Answering. *arXiv preprint arXiv:2511.20086*. Accepted at the 41st ACM/SIGAPP Symposium on Applied Computing (SAC 2026).

Haochun Wang, Sendong Zhao, Zewen Qiang, Nuwa Xi, Bing Qin, and Ting Liu. 2025. LLMs May Perform MCQA by Selecting the Least Incorrect Option. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5852–5862, Abu Dhabi, UAE. Association for Computational Linguistics.

Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. 2024. Large Language Models Are Not Robust Multiple Choice Selectors. In *The Twelfth International Conference on Learning Representations*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, volume 36.

Appendix

A Prompt Templates

All prompts are stored under `prompts/{version}/` and snapshotted into each run directory for reproducibility. Curly-brace tokens denote substitution slots filled at runtime.

Baseline, Cyclic Permutation, and PriDe (`v1/direct_mcq.txt`). Used by *baseline*, *cyclic*, and *pride*. The same single prompt is issued for every call; cyclic permutation rotates the content assigned to each label across four calls, and PriDe issues four logprob-mode calls at calibration time plus one at inference time.

```
Answer the following multiple-choice question.

Question: {question}

Options:
A. {option_a}
B. {option_b}
C. {option_c}
D. {option_d}

Respond with only the letter.
```

Two-Stage Stage 1: Free-Text Elicitation (`v1/free_text.txt`). The question is presented without options so the model cannot anchor on a label. The raw completion is passed verbatim to Stage 2.

```
Answer the following question based on your
knowledge.

Question: {question}

Respond with a short direct answer only.
```

Two-Stage Stage 2: Option Matching (`v1/option_matching.txt`). The Stage 1 free-text answer is injected as `{free_text}`. This prompt is the primary second-stage extraction prompt used by the main two-stage method.

```
You are given a question, a reference answer,
and four options.

Question: {question}

Reference answer: {free_text}
```

```
Options:
A. {option_a}
B. {option_b}
C. {option_c}
D. {option_d}

Select the option that best matches the reference
answer in the context of the question. If the
reference answer is imperfect or incomplete,
choose the closest option. Respond with only
the letter.
```

v2 Stage 1 Ablation: Detail-Elicitation (`v2/free_text.txt`). Replaces v1 Stage 1; instructs the model to include distinguishing detail and suppresses chain-of-thought. Stage 2 is unchanged from v1.

```
Answer the following question based on your
knowledge.

Question: {question}

Respond with a concise answer phrase. Include
enough detail to distinguish the answer from
similar alternatives. Do not explain your
reasoning.
```

v3 Stage 2 Ablation: Semantic Option Matching (`v3/option_matching.txt`). Replaces v1 Stage 2; adds explicit guidance to prioritise meaning over surface wording and to handle incomplete or ambiguously phrased free-text answers. Stage 1 is unchanged from v1.

```
You are given a question, a reference answer,
and four options.

Question: {question}

Reference answer: {free_text}

Options:
A. {option_a}
B. {option_b}
C. {option_c}
D. {option_d}

Choose the option that is semantically closest to
the  reference answer in the context of the
question. Prioritize meaning over exact wording.
If the reference answer is incomplete, ambiguous,
or phrased differently from the options, choose
the option most consistent with it. Respond with
only  A, B, C, or D.
```

Text Extraction Stage 1 (`v1/text_extraction.txt`). Used by *text_extraction*. Unlike the two-stage free-text prompt, options are visible so the model can identify the correct answer text precisely; however, the model is forbidden from outputting the letter. Matching back to an option uses the same cascade as *twostage_semantic_match* (exact match, then substring containment, then embedding cosine similarity via the Apache-2.0-licensed `all-MiniLM-L6-v2` model at a 0.30 threshold; `src/twoprompt/parsing/text_matcher.py`), with no second LLM call. (`rapidfuzz` fuzzy matching is used only by an offline diagnostic script, `scripts/evaluate_run.py`'s free-text-vs-gold decomposition report, and never in the runtime scoring path.)

```
Answer the following question. Read all options
carefully.

Question: {question}

Options:
A. {option_a}
B. {option_b}
C. {option_c}
D. {option_d}

Respond with the correct answer text only. Do not
write the option letter. Do not explain.
```

B PriDe Implementation Details

PriDe (Zheng et al., 2024) estimates a model's positional bias prior from a held-out calibration set and uses it to debias per-question predictions. The implementation is evaluated on three models that expose per-token log-probabilities: `Qwen/Qwen2.5-7B-Instruct-Turbo` (Together AI API), `Qwen/Qwen2.5-7B-Instruct` (local), and `meta-llama/Llama-3.1-8B-Instruct` (local).

Logprob extraction. When `request_logprobs=True`, the client appends an assistant-turn prefill of `"The answer is "` before issuing the request. Because the model continues generation from this prefix, its first generated token is almost always a bare letter (A, B, C, or D), so the letter log-probabilities appear directly in position 0 of the returned token sequence. The implementation requests `top_logprobs=20` to maximise coverage of the four option letters.

The function `merge_option_logprobs` normalises the raw response into a `dict[str, float]` mapping each option letter to its highest observed log-probability. It handles two logprob response formats transparently:

- **Standard OpenAI format**: the `logprobs.content` field is a list of token objects, each carrying a `top_logprobs` list of `(token, logprob)` pairs.
- **Together AI non-standard format**: the `logprobs` object carries three parallel arrays — `tokens`, `token_logprobs`, and

`top_logprobs` (a list of `{token: logprob}` dicts per position) — with an empty `content` list.

Token strings are stripped and upper-cased before lookup; tokens that do not resolve to one of A, B, C, or D are discarded. Any letter absent from the `top_logprobs` response is assigned a floor value of -30.0 so that the downstream softmax can still produce a valid four-class distribution.

Phase 1: Calibration. Before processing any evaluation questions, `PriDeRunner` draws its $K = 50$ calibration questions from a pool that has already had every evaluation-split question ID removed — disjointness is guaranteed *by construction*, not merely by a seeded shuffle that happens not to collide. `scripts/run_experiment.py`'s `load_calibration_questions` performs this filtering before any sampling occurs:

```
def load_calibration_questions(benchmark,
    eval_question_ids, paths):
    """Return questions from the full normalized
    CSV that are NOT in the eval split. Used by
    PriDeRunner so the position prior is
    ↪  estimated
    on questions the runner will never be scored
    on, eliminating calibration/eval overlap."""
    ...
    df = df[~df["question_id"].isin(eval_
    question_ids)].drop_duplicates(
        subset="question_id"
    )
```

The $K = 50$ calibration questions (seed = 42) are then sampled from this already-disjoint pool, so no eval question can ever be selected for calibration, regardless of seed. For each calibration question, four API calls are made — one per cyclic rotation of the option texts — each with `request_logprobs=True`. The responses are parsed to yield a 4×4 probability matrix M where $M_{k,j}$ is the softmax-normalised probability that the model chooses letter j under cyclic permutation k. The per-question positional prior is estimated as:

$$\hat{P}_{\text{prior}}(d_i) = \text{softmax}\left(\frac{1}{|\mathcal{I}|} \sum_{I \in \mathcal{I}} \log P_{\text{obs}}(d_i \mid q, x^I) \right).$$

Per-question prior vectors are averaged and renormalised to obtain the global prior $\hat{P}_{\text{eprior}}$, which is cached to a JSON sidecar file for reproducibility.

Phase 2: Transfer Debiasing. For each evaluation question, one standard `request_logprobs=True` call is made using the baseline prompt. The debiased distribution is:

$$P_{\text{debiased}}(o_i \mid q, x) \;\propto\; \frac{P_{\text{obs}}(d_i \mid q, x)}{\hat{P}_{\text{eprior}}(d_i)},$$

computed as an elementwise ratio clipped at $\epsilon = 10^{-12}$ and renormalised. The argmax of P_{debiased} is the final predicted letter.

C Experimental Configuration

Table 1 reports the per-model configuration used across the main evaluation. All models share temperature $= 0.0$ and seed $= 42$. `max_tokens` is 500 for every condition except *independent_hypothesis*, the sole condition that overrides it to 4000 (Appendix D); Table 1 reports the shared 500 value used by the main results. API models are called asynchronously; concurrency, minimum inter-call delay, and retry settings are set per provider to stay within rate limits. Local models (`Qwen/Qwen2.5-7B-Instruct` and `meta-llama/Llama-3.1-8B-Instruct`) are run from a sibling repository via a HuggingFace Transformers `AutoModelForCausalLM` backend, as SLURM batch jobs on an institutional HPC cluster. The two local models were run on single NVIDIA A100 MIG 3g.40gb slices (40,GB VRAM). Producing the reported local-model results consumed approximately 128 GPU-hours in total across both models, both benchmarks, and all conditions. API-model inference does not incur local compute and is not included in this figure. Both local models run synchronously, so concurrency and delay settings do not apply to them.

Local-model inference used Python 3.10.5, PyTorch 2.12.0, transformers 5.9.0, and sentence-transformers 5.5.1 (with `all-MiniLM-L6-v2` at revision `1110a243`). Statistical analysis used statsmodels 0.14.6 (McNemar tests), rapidfuzz 3.14.5, numpy, and scipy.

D Independent Hypothesis Implementation

The independent-hypothesis condition (*independent_hypothesis*, `src/twoprompt/runners/independent_hypothesis.py`) evaluates each option as an isolated hypothesis rather than presenting all options together.

Prompt template (`prompts/v1/independent_hypothesis.txt`), verbatim.

Table 1: Per-model configuration for all experiments. Dashes indicate settings that do not apply to local synchronous inference.

Model	Provider	Conc.	Delay (s)	Retries	Temp.	Tokens	Seed
gpt-4.1-mini	OpenAI	10	0.0	3	0.0	500	42
gemini-2.5-flash	Google	1	1.5	5	0.0	500	42
llama-3.1-8b-instant	Groq	2	0.5	3	0.0	500	42
Qwen2.5-7B-Instruct-Turbo	Together AI	5	0.2	3	0.0	500	42
Qwen2.5-7B-Instruct	Local (HF)	—	—	—	0.0	500	42
Llama-3.1-8B-Instruct	Local (HF)	—	—	—	0.0	500	42

```
Question: {question}

Hypothesis: The correct answer is {option_text}.

Task: Please evaluate whether this hypothesis
correctly and accurately answers the question.
First, provide a brief step-by-step analysis.
Then, output a final confidence score between
0 and 100 indicating the probability that this
hypothesis is the true answer. Strictly format
your final score within tags, exactly like
this: <score>X</score>.
```

Score format and parsing. The model is asked for a 0–100 confidence score wrapped in `<score>X</score>` tags. Extraction uses a dedicated regex, case-insensitive, with last-occurrence-wins semantics (consistent with the rest of the codebase's parser, which prefers a model's final restated answer over an earlier draft):

```
_SCORE_PATTERN = re.compile
(r"<score>\s*(-?\d+(?:\.\d+)?)\s*</score>",
                              re.IGNORECASE)
```

On parse failure the option's score is set to 0.0 and the option is marked `parse_ok = False`, but it still participates in the argmax (a confidence of 0 is not treated as "missing").

Calls per question. One parallel API call per *real* option – 4 for ordinary questions, 3 for the three ARC-Challenge questions whose fourth option is genuinely absent from the source data. No option is ever shown alongside another in the same prompt, and no call is made for an option that is missing (NaN or empty string).

Aggregation and tie-break. The final prediction is the argmax of the per-option confidence scores. Exact ties are broken by an RNG seeded from the run seed and the question ID (not a shared generator), so the outcome is reproducible independent of async completion order:

```
best_score = max(scores.values())
tied = sorted(letter for letter, s in scores
.items()
if s == best_score)
if len(tied) == 1:
    return tied[0]
rng = random.Random(f"{seed}:{question_id}")
return rng.choice(tied)
```

`max_tokens` override. This condition runs with `max_tokens: 4000` instead of the project-wide default of 500 (Table 1), set in `config/independent_hypothesis.yaml`. The override exists because Gemini 2.5 Flash's internal "thinking" tokens draw from the same `max_output_tokens` budget as its visible response; at 500 tokens it exhausted the budget on hidden reasoning before emitting the `<score>` tag on 976/1000 rows (`finish_reason = MAX_TOKENS`), collapsing its accuracy to chance level on the initial run. The override is shared across all four API jobs in this config (the pipeline has no per-model `max_tokens`), so it also gives the other three models more headroom, though they were not hitting the cap before the change.

Tie rates. Per-model × benchmark tie rates – the fraction of scored questions where ≥2 real options share the maximum confidence score – are reported in Table 13. Across the six models and two benchmarks (Gemini 2.5 Flash × ARC-Challenge excluded, above), tie rates range from 6.2% (GPT-4.1 mini, ARC-Challenge) to 43.2% (Llama 3.1 8B local, MMLU).

E Fourth Diagnostic Cell: Visible + LLM Matcher

Of the 2 × 2 Stage-1-visibility × Stage-2-matcher-type design (hidden/visible options × embedding/LLM matching), three cells are produced directly by this repository (*two_prompt* = hidden+LLM, *twostage_semantic_match* = hidden+embedding, *text_extraction* = visible+embedding). The fourth cell, *visible_llm_matcher* (visible options, LLM-based matching), is deliberately excluded from `ALL_METHODS` in `src/twoprompt/config/`

`experiment.py`: this repository has no runner that can launch it. Its runner lives in a separate experiment worktree, `experiments/visible_llm_matcher/`, built on top of the project's shared MCQ-evaluation infrastructure.

Stage 1: reused, not regenerated. The experiment's Stage-1 source loader enforces (fails closed rather than silently substituting) that Stage-1 free-text completions are the *same* completions already produced by the *text_extraction* condition – no second Stage-1 sample is generated. For 10 of the twelve model-benchmark cells this source is read directly from this repository's own `paper_results/eval_ready/{paper_api_main,paper_local_main}/`. The exception is the two local models on ARC-Challenge: this repository's own local ARC *text_extraction* files were stale at 850 rows at the time, so that one source is instead read from a corrected, complete 1000-row rerun in the sibling model-generalization repository.

Stage 2: real LLM call, prompt template. Stage 2 makes a genuine model call (not a heuristic) using the following template, injecting the reused Stage-1 free-text answer:

```
You are given a question, a reference answer,
and four options.

Question: {question}

Reference answer: {free_text}

Options:
A. {option_a}
B. {option_b}
C. {option_c}
D. {option_d}

Select the option that best matches the reference
answer in the context of the question. If the
reference answer is imperfect or incomplete,
choose the closest option. Respond with only
the letter.
```

Coverage. All 6 models $\times$ 2 benchmarks, 1000 rows each, no exceptions.

F Flip-Rate Protocol

To measure option-order sensitivity directly, we computed a per-question "any-flip" rate for three conditions: baseline, `two_prompt`, and `twostage_semantic_match` (Table 8).

Coverage: six models, both benchmarks. Flip rate and RStd are both reported for all six models across both benchmarks, under baseline and `two_prompt` (flip rate) and under baseline, `two_prompt`, and `twostage_semantic_match` (RStd). Coverage is not uniform: all 12 semantic-matching cells (both benchmarks, all six models) score below 90% of their 1,000 questions, ranging from 68.8% (Llama-local, MMLU) to 88.8% (GPT-4.1 mini, ARC-Challenge). Among baseline/`two_prompt` cells, only Gemini's MMLU `two_prompt` cell falls below 90%, at 83.3%; its ARC-Challenge `two_prompt` cell is scored at 93.0% and is not part of this low-coverage group. All other baseline/`two_prompt` cells score at or above 90%.

Permutation count and the three-option ARC questions. Each question is scored under n cyclic rotations of its option order, $n = 4$ normally and $n = 3$ for the three ARC-Challenge questions whose fourth option is genuinely absent from the source data – no synthetic fourth option is fabricated for these three questions in this analysis.

Flip definition and denominator. A question "flips" if its parsed answer takes ≥ 2 distinct values across its n permutation rows. The denominator for a per-question flip determination is that question's own permutation count ($n = 3$ or $n = 4$); the reported flip *rate* is the fraction of the benchmark's 1000 questions that flip.

Stage reuse for *two_prompt*. Stage 1 (free-text) completions were reused verbatim from the existing *two_prompt* run for both models – Stage 1 does not depend on option order, so no new Stage-1 calls were needed. Only Stage 2 (option matching, which is order-sensitive under permutation) was re-run with new inference calls, one per (question, permutation).

Why cyclic is not reported separately. The *cyclic* condition has no flip rate of its own. Its per-permutation predictions are exactly the *baseline* method's predictions under each rotation, since both issue the same single-call direct-MCQ prompt with identical settings; and its method output is a single majority-voted answer per question, which cannot flip by construction. Table 8 therefore omits a Cyclic column.

G Tie-Break Seed Sensitivity

What was done. The independent-hypothesis condition's final prediction depends on a seeded random tie-break only when two or more op-

tions are exactly tied at the maximum confidence score (Appendix D). To check how sensitive reported accuracy is to that seed, we recomputed `final_prediction` for all 11 available independent-hypothesis eval-ready files (Gemini × ARC-Challenge excluded – extensive provider-side API failures, 68.7% of rows) under 10 alternative tie-break seeds, $0, 1, \ldots, 9$, all distinct from the original run seed 42. No new model inference was performed: the recomputation reruns only the argmax-with-tiebreak function (Appendix D) over the four (or three) per-option confidence scores already stored in each row of the eval-ready CSVs. As a validation step, recomputing with the original seed (42) reproduced the stored `final_prediction` for 1000/1000 rows in all 11 files, and the recomputed accuracy at seed 42 matched the cached end-to-end accuracy exactly in every cell, confirming the tie-break implementation used for the sweep is faithful to the original runner before drawing conclusions from it. The original seed (42) is excluded from the reported mean/sd/min/max in Table 13; only the 10 new seeds contribute to those statistics.

H Fallback Analysis

As a diagnostic (not a main result), we ask: what accuracy would we observe if every unscorable output (a successful API/model call that nonetheless failed to yield a parseable answer) were replaced by that same model's *baseline* prediction on the same question? This isolates how much of a condition's accuracy loss is attributable to the matching/extraction stage specifically, versus the underlying model's competence on the question.

Substitution rule. For each unscorable row, the substituted value is *that model's own baseline-condition prediction for the same question* – not a fixed constant (e.g. not "always incorrect" or a random guess). If no matching baseline row exists for that model and question, the row is left unscorable and counted separately.

Coverage. Fallback re-scoring applies only to methods with a free-text/extraction stage that can fail to produce a parseable letter: *two_prompt* (v1/v2/v3), *text_extraction*, and *twostage_semantic_match* (v1/v2). It explicitly does *not* apply to *cyclic* (majority vote over four permutations has different unscorable semantics – "unscorable" there means all permutations failed, not a single extraction miss), *pride* (logprob-based scoring, no free-text stage to fail), *baseline*, or *independent_hypothesis*.

I Full Results Tables

This appendix reports the complete results referenced throughout Section 4: the MMLU tables not shown in the main text, and the ARC-Challenge counterparts of every table. The patterns discussed in the main text hold on both benchmarks unless stated otherwise.

Table 2: End-to-end accuracy (%) with 95% Clopper–Pearson confidence intervals on MMLU. Rows are methods, columns are models. PriDe requires logprob access and is evaluated only for the three models with that access; – indicates not evaluated.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Baseline	81.8 [79.3–84.1]	84.9 [82.5–87.1]	65.2 [62.2–68.2]	50.2 [47.1–53.3]	68.9 [65.9–71.8]	67.1 [64.1–70.0]
Two-stage	80.1 [77.5–82.5]	68.3 [65.3–71.2]	64.9 [61.9–67.9]	49.3 [46.2–52.4]	67.3 [64.3–70.2]	64.9 [61.9–67.9]
Cyclic	81.5 [79.0–83.9]	86.9 [84.6–88.9]	66.6 [63.6–69.5]	57.0 [53.9–60.1]	69.8 [66.8–72.6]	69.4 [66.4–72.2]
PriDe	–	–	–	44.2 [41.1–47.3]	70.0 [67.1–72.8]	67.0 [64.0–69.9]
Text extraction	81.7 [79.2–84.1]	88.0 [85.8–89.9]	62.3 [59.2–65.3]	45.7 [42.6–48.8]	59.3 [56.2–62.4]	63.5 [60.4–66.5]
Semantic matching	45.9 [42.8–49.0]	48.9 [45.8–52.0]	36.1 [33.1–39.2]	32.0 [29.1–35.0]	38.9 [35.9–42.0]	36.5 [33.5–39.6]
Independent hypothesis	83.0 [80.5–85.3]	83.4 [80.9–85.7]	60.5 [57.4–63.5]	51.2 [48.1–54.3]	66.3 [63.3–69.2]	62.7 [59.6–65.7]
Visible + LLM matcher	82.4 [79.9–84.7]	84.4 [82.0–86.6]	64.2 [61.1–67.2]	56.7 [53.6–59.8]	69.7 [66.7–72.5]	66.4 [63.4–69.3]

Table 3: End-to-end accuracy (%) with 95% Clopper–Pearson confidence intervals on ARC-Challenge. Rows are methods, columns are models. PriDe requires logprob access and is evaluated only for the three models with that access; – indicates not evaluated.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Baseline	96.1 [94.7–97.2]	96.9 [95.6–97.9]	82.2 [79.7–84.5]	58.0 [54.9–61.1]	90.1 [88.1–91.9]	87.8 [85.6–89.8]
Two-stage	94.3 [92.7–95.7]	86.6 [84.3–88.7]	79.1 [76.4–81.6]	58.3 [55.2–61.4]	84.6 [82.2–86.8]	79.7 [77.1–82.2]
Cyclic	96.1 [94.7–97.2]	97.0 [95.7–98.0]	84.0 [81.6–86.2]	72.9 [70.0–75.6]	90.8 [88.8–92.5]	90.5 [88.5–92.2]
PriDe	–	–	–	48.8 [45.7–51.9]	90.3 [88.3–92.1]	89.1 [87.0–91.0]
Text extraction	95.6 [94.1–96.8]	96.3 [94.9–97.4]	82.7 [80.2–85.0]	59.1 [56.0–62.2]	85.5 [83.2–87.6]	87.3 [85.1–89.3]
Semantic matching	52.1 [49.0–55.2]	55.4 [52.3–58.5]	43.9 [40.8–47.0]	36.9 [33.9–40.0]	47.8 [44.7–50.9]	45.9 [42.8–49.0]
Independent hypothesis	94.3 [92.7–95.7]	–	81.1 [78.5–83.5]	72.4 [69.5–75.2]	85.8 [83.5–87.9]	82.3 [79.8–84.6]
Visible + LLM matcher	96.1 [94.7–97.2]	96.5 [95.2–97.6]	82.9 [80.4–85.2]	70.4 [67.5–73.2]	90.5 [88.5–92.2]	87.9 [85.7–89.9]

Independent hypothesis × Gemini 2.5 Flash on ARC-Challenge is excluded (–): the run faced provider-side API failures (68.7% API failures), so the cell was never completed and is omitted from canonical evaluation rather than reported with corrupted numbers.

Table 4: End-to-end vs. conditional accuracy (%) on MMLU, for the four methods that produce unscorable outputs. End-to-end counts unscorable outputs as incorrect; conditional accuracy is computed over scorable outputs only.

Method	**Model**	**Scored/Total**	**End-to-end**	**Conditional**
Two-stage	GPT-4.1 mini	1000/1000	80.1	80.1
Two-stage	Gemini 2.5 Flash	833/1000	68.3	82.0
Two-stage	Llama 3.1 8B (API)	997/1000	64.9	65.1
Two-stage	Llama 3.1 8B (local)	948/1000	49.3	52.0
Two-stage	Qwen 2.5 7B (API)	999/1000	67.3	67.4
Two-stage	Qwen 2.5 7B (local)	1000/1000	64.9	64.9
Semantic matching	GPT-4.1 mini	849/1000	45.8	53.9
Semantic matching	Gemini 2.5 Flash	848/1000	48.8	57.5
Semantic matching	Llama 3.1 8B (API)	783/1000	36.1	46.1
Semantic matching	Llama 3.1 8B (local)	688/1000	32.0	46.5
Semantic matching	Qwen 2.5 7B (API)	854/1000	38.8	45.4
Semantic matching	Qwen 2.5 7B (local)	809/1000	36.4	45.0
Text extraction	GPT-4.1 mini	998/1000	81.7	81.9
Text extraction	Gemini 2.5 Flash	998/1000	88.0	88.2
Text extraction	Llama 3.1 8B (API)	995/1000	62.3	62.6
Text extraction	Llama 3.1 8B (local)	851/1000	45.7	53.7
Text extraction	Qwen 2.5 7B (API)	861/1000	59.3	68.9
Text extraction	Qwen 2.5 7B (local)	954/1000	63.5	66.6
Visible + LLM matcher	GPT-4.1 mini	1000/1000	82.4	82.4
Visible + LLM matcher	Gemini 2.5 Flash	953/1000	84.4	88.6
Visible + LLM matcher	Llama 3.1 8B (API)	1000/1000	64.2	64.2
Visible + LLM matcher	Llama 3.1 8B (local)	961/1000	56.7	59.0
Visible + LLM matcher	Qwen 2.5 7B (API)	1000/1000	69.7	69.7
Visible + LLM matcher	Qwen 2.5 7B (local)	1000/1000	66.4	66.4

Table 5: End-to-end vs. conditional accuracy (%) on ARC-Challenge, for the four methods that produce unscorable outputs. End-to-end counts unscorable outputs as incorrect; conditional accuracy is computed over scorable outputs only.

Method	**Model**	**Scored/Total**	**End-to-end**	**Conditional**
Two-stage	GPT-4.1 mini	1000/1000	94.3	94.3
Two-stage	Gemini 2.5 Flash	930/1000	86.6	93.1
Two-stage	Llama 3.1 8B (API)	1000/1000	79.1	79.1
Two-stage	Llama 3.1 8B (local)	941/1000	58.3	62.0
Two-stage	Qwen 2.5 7B (API)	1000/1000	84.6	84.6
Two-stage	Qwen 2.5 7B (local)	1000/1000	79.7	79.7
Semantic matching	GPT-4.1 mini	888/1000	52.1	58.7
Semantic matching	Gemini 2.5 Flash	865/1000	55.4	64.0
Semantic matching	Llama 3.1 8B (API)	858/1000	43.9	51.2
Semantic matching	Llama 3.1 8B (local)	769/1000	36.9	48.0
Semantic matching	Qwen 2.5 7B (API)	880/1000	47.8	54.3
Semantic matching	Qwen 2.5 7B (local)	862/1000	45.9	53.2
Text extraction	GPT-4.1 mini	1000/1000	95.6	95.6
Text extraction	Gemini 2.5 Flash	998/1000	96.3	96.5
Text extraction	Llama 3.1 8B (API)	1000/1000	82.7	82.7
Text extraction	Llama 3.1 8B (local)	943/1000	59.1	62.7
Text extraction	Qwen 2.5 7B (API)	949/1000	85.5	90.1
Text extraction	Qwen 2.5 7B (local)	993/1000	87.3	87.9
Visible + LLM matcher	GPT-4.1 mini	1000/1000	96.1	96.1
Visible + LLM matcher	Gemini 2.5 Flash	997/1000	96.5	96.8
Visible + LLM matcher	Llama 3.1 8B (API)	1000/1000	82.9	82.9
Visible + LLM matcher	Llama 3.1 8B (local)	967/1000	70.4	72.8
Visible + LLM matcher	Qwen 2.5 7B (API)	1000/1000	90.5	90.5
Visible + LLM matcher	Qwen 2.5 7B (local)	1000/1000	87.9	87.9

Table 6: Recall standard deviation (RStd, pp) with 95% bootstrap confidence intervals (10,000 resamples) on MMLU, for the three methods with recomputed CIs (baseline, two-stage, and semantic matching). Lower values indicate more uniform recall across answer positions. Rows are methods, columns are models.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Baseline	1.4 [0.7–4.5]	0.7 [0.5–3.4]	5.2 [2.9–8.5]	10.2 [7.6–13.3]	4.2 [2.0–7.5]	8.1 [5.3–11.3]
Two-stage	1.3 [0.7–4.5]	4.5 [2.6–7.2]	3.3 [1.5–6.7]	8.1 [5.5–11.5]	1.9 [0.9–5.5]	4.3 [2.1–7.5]
Semantic matching	3.6 [1.6–7.4]	4.4 [2.2–8.2]	3.5 [1.6–7.5]	1.6 [1.0–6.5]	2.1 [1.1–6.2]	3.0 [1.4–7.0]

Table 7: Recall standard deviation (RStd, pp) with 95% bootstrap confidence intervals (10,000 resamples) on ARC-Challenge, for the three methods with recomputed CIs (baseline, two-stage, and semantic matching). Lower values indicate more uniform recall across answer positions. Rows are methods, columns are models.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Baseline	0.2 [0.3–1.9]	0.9 [0.4–2.1]	1.8 [0.8–4.9]	12.5 [10.0–15.4]	1.7 [0.7–3.9]	4.7 [2.6–7.3]
Two-stage	0.8 [0.4–2.7]	2.6 [1.4–4.4]	4.0 [2.2–6.7]	8.0 [5.3–11.2]	4.6 [3.0–6.9]	3.1 [1.4–5.8]
Semantic matching	1.6 [0.9–5.8]	1.0 [0.8–5.3]	3.1 [1.4–7.0]	3.1 [1.3–7.3]	2.8 [1.2–6.8]	1.7 [0.9–6.0]

Table 8: Flip rate (%): fraction of questions whose parsed answer changes across the four cyclic rotations, for baseline and two-stage prompting across all six models and both benchmarks. Semantic matching's flip rate is 0.0% for every model on both benchmarks and is omitted from this table. The Cyclic condition is not shown as a row: its flip rate is identical to Baseline by construction, since both are computed from the same underlying permutation trace before/after majority voting.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
MMLU						
Baseline	21.6	13.9	41.0	80.3	31.6	41.3
Two-stage	11.8	15.4	36.3	63.1	32.6	46.5
ARC-Challenge						
Baseline	6.0	2.7	21.3	74.6	12.9	17.6
Two-stage	5.3	7.3	23.7	55.8	19.0	29.8

Table 9: Broken/fixed question counts relative to baseline on MMLU, with McNemar asymptotic test p-values. Broken = baseline correct, method incorrect; fixed = baseline incorrect, method correct. Cells show broken/fixed (McNemar p). All cells with a comparison have a McNemar value; – marks methods not evaluated for that model (PriDe: no logprob access)

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Two-stage	79/62 (p=0.178)	87/23 (p<.001)	118/115 (p=0.896)	205/218 (p=0.560)	99/84 (p=0.301)	126/104 (p=0.166)
Cyclic	18/15 (p=0.728)	12/16 (p=0.571)	26/40 (p=0.110)	82/148 (p<.001)	26/35 (p=0.306)	28/51 (p=0.013)
PriDe	–	–	–	219/157 (p=0.002)	13/24 (p=0.100)	23/22 (p=1.000)
Text extraction	46/46 (p=0.917)	31/33 (p=0.901)	106/80 (p=0.067)	162/170 (p=0.701)	45/42 (p=0.830)	68/50 (p=0.118)
Semantic matching	289/41 (p<.001)	276/20 (p<.001)	258/79 (p<.001)	166/115 (p=0.003)	286/74 (p<.001)	270/68 (p<.001)
Independent hypothesis	95/102 (p=0.669)	97/43 (p<.001)	176/156 (p=0.297)	187/234 (p=0.025)	178/136 (p=0.021)	170/137 (p=0.068)
Visible + LLM matcher	40/46 (p=0.590)	27/31 (p=0.694)	86/76 (p=0.480)	147/232 (p<.001)	34/42 (p=0.422)	58/51 (p=0.565)

Table 10: Broken/fixed question counts relative to baseline on ARC-Challenge, with McNemar asymptotic test p-values. Broken = baseline correct, method incorrect; fixed = baseline incorrect, method correct. Cells show broken/fixed (McNemar p). All cells with a comparison have a McNemar value; – marks methods not evaluated for that model (PriDe: no logprob access) or the excluded independent-hypothesis/Gemini/ARC-Challenge cell.

Method	GPT-4.1 mini	Gemini 2.5 Flash	Llama 3.1 8B (API)	Llama 3.1 8B (local)	Qwen 2.5 7B (API)	Qwen 2.5 7B (local)
Two-stage	34/16 (p=0.016)	43/4 (p<.001)	100/69 (p=0.021)	174/210 (p=0.074)	91/36 (p<.001)	134/53 (p<.001)
Cyclic	5/5 (p=0.752)	3/1 (p=0.617)	9/27 (p=0.005)	55/201 (p<.001)	6/13 (p=0.169)	10/37 (p<.001)
PriDe	–	–	–	234/140 (p<.001)	4/6 (p=0.752)	8/21 (p=0.026)
Text extraction	14/9 (p=0.404)	10/2 (p=0.043)	45/50 (p=0.682)	178/209 (p=0.127)	13/13 (p=0.845)	35/35 (p=0.905)
Semantic matching	346/11 (p<.001)	286/5 (p<.001)	323/48 (p<.001)	220/127 (p<.001)	352/27 (p<.001)	341/34 (p<.001)
Independent hypothesis	36/24 (p=0.156)	–	99/98 (p=1.000)	131/270 (p<.001)	95/52 (p<.001)	116/64 (p<.001)
Visible + LLM matcher	8/8 (p=0.803)	8/2 (p=0.114)	45/52 (p=0.542)	118/258 (p<.001)	9/13 (p=0.522)	36/37 (p=1.000)

Independent hypothesis × Gemini 2.5 Flash on ARC-Challenge is excluded (–): the run faced provider-side API failures (68.7% API failures), so the cell was never completed and is omitted from canonical evaluation rather than reported with corrupted numbers.

Table 11: The 2×2 Stage-1/Stage-2 diagnostic grid on MMLU: end-to-end accuracy (%) with 95% CI, crossing whether Stage 1 hides or shows the answer options against whether Stage 2 matching uses an LLM call or an embedding similarity. Baseline (single-call, no decomposition) is shown for reference.

Model	Baseline	Hidden + LLM	Hidden + Embed.	Visible + Embed.	Visible + LLM
GPT-4.1 mini	81.8 [79.3–84.1]	80.1 [77.5–82.5]	45.8 [42.7–48.9]	81.7 [79.2–84.1]	82.4 [79.9–84.7]
Gemini 2.5 Flash	84.9 [82.5–87.1]	68.3 [65.3–71.2]	48.8 [45.7–51.9]	88.0 [85.8–89.9]	84.4 [82.0–86.6]
Llama 3.1 8B (API)	65.2 [62.2–68.2]	64.9 [61.9–67.9]	36.1 [33.1–39.2]	62.3 [59.2–65.3]	64.2 [61.1–67.2]
Llama 3.1 8B (local)	50.2 [47.1–53.3]	49.3 [46.2–52.4]	32.0 [29.1–35.0]	45.7 [42.6–48.8]	56.7 [53.6–59.8]
Qwen 2.5 7B (API)	68.9 [65.9–71.8]	67.3 [64.3–70.2]	38.8 [35.8–41.9]	59.3 [56.2–62.4]	69.7 [66.7–72.5]
Qwen 2.5 7B (local)	67.1 [64.1–70.0]	64.9 [61.9–67.9]	36.4 [33.4–39.5]	63.5 [60.4–66.5]	66.4 [63.4–69.3]

Table 12: The 2×2 Stage-1/Stage-2 diagnostic grid on ARC-Challenge: end-to-end accuracy (%) with 95% CI, crossing whether Stage 1 hides or shows the answer options against whether Stage 2 matching uses an LLM call or an embedding similarity. Baseline (single-call, no decomposition) is shown for reference.

Model	Baseline	Hidden + LLM	Hidden + Embed.	Visible + Embed.	Visible + LLM
GPT-4.1 mini	96.1 [94.7–97.2]	94.3 [92.7–95.7]	52.1 [49.0–55.2]	95.6 [94.1–96.8]	96.1 [94.7–97.2]
Gemini 2.5 Flash	96.9 [95.6–97.9]	86.6 [84.3–88.7]	55.4 [52.3–58.5]	96.3 [94.9–97.4]	96.5 [95.2–97.6]
Llama 3.1 8B (API)	82.2 [79.7–84.5]	79.1 [76.4–81.6]	43.9 [40.8–47.0]	82.7 [80.2–85.0]	82.9 [80.4–85.2]
Llama 3.1 8B (local)	58.0 [54.9–61.1]	58.3 [55.2–61.4]	36.9 [33.9–40.0]	59.1 [56.0–62.2]	70.4 [67.5–73.2]
Qwen 2.5 7B (API)	90.1 [88.1–91.9]	84.6 [82.2–86.8]	47.8 [44.7–50.9]	85.5 [83.2–87.6]	90.5 [88.5–92.2]
Qwen 2.5 7B (local)	87.8 [85.6–89.8]	79.7 [77.1–82.2]	45.9 [42.8–49.0]	87.3 [85.1–89.3]	87.9 [85.7–89.9]

Table 13: Independent hypothesis (IHS) detail: end-to-end accuracy, delta vs. baseline (pp), tie-break rate (fraction of scored questions where $\geq$2 options shared the maximum confidence score), accuracy spread across 10 alternative tie-break seeds (mean/sd/min/max, original seed 42 excluded from these four columns), and recall standard deviation (RStd, pp) with 95% bootstrap CI (10,000 resamples). Gemini 2.5 Flash $\times$ ARC-Challenge is excluded (–): only 313 of 1,000 questions returned successfully due to provider-side API failures, so the cell is shown as excluded rather than silently dropped or reported from partial data (see Table 3).

Model	Benchmark	IHS Acc.	Δ (pp)	Tie rate	Seed mean	Seed sd	Seed min	Seed max	RStd
GPT-4.1 mini	ARC-Challenge	94.3	-1.8	6.2	94.4	0.39	93.8	95.0	1.8 [0.7–3.6]
GPT-4.1 mini	MMLU	83.0	+1.2	14.7	82.6	0.65	81.7	83.9	2.4 [1.0–5.1]
Gemini 2.5 Flash	ARC-Challenge	–	–	–	–	–	–	–	–
Gemini 2.5 Flash	MMLU	83.4	-1.5	15.8	82.5	0.41	81.7	83.0	2.9 [1.4–5.4]
Llama 3.1 8B (API)	ARC-Challenge	81.1	-1.1	16.4	81.4	0.81	80.0	82.7	1.1 [0.7–4.3]
Llama 3.1 8B (API)	MMLU	60.5	-4.7	29.8	62.3	0.64	61.2	63.1	2.4 [1.1–6.0]
Qwen 2.5 7B (API)	ARC-Challenge	85.8	-4.3	12.4	85.3	0.36	84.6	85.8	2.1 [0.9–4.6]
Qwen 2.5 7B (API)	MMLU	66.3	-2.6	25.9	66.6	0.61	65.6	67.4	3.3 [1.5–6.7]
Qwen 2.5 7B (local)	ARC-Challenge	82.3	-5.5	12.6	82.3	0.59	81.4	83.1	1.9 [0.8–4.8]
Qwen 2.5 7B (local)	MMLU	62.7	-4.4	23.5	63.4	0.72	61.9	64.4	3.9 [1.8–7.2]
Llama 3.1 8B (local)	ARC-Challenge	72.4	+14.4	27.7	72.5	0.65	71.6	73.2	2.2 [1.0–5.4]
Llama 3.1 8B (local)	MMLU	51.2	+1.0	43.2	52.6	0.88	51.6	54.0	3.1 [1.3–6.8]

Table 14: Fallback re-scoring on MMLU: original end-to-end accuracy vs. accuracy if unscorable outputs fall back to the baseline prediction, per model $\times$ method. Independent hypothesis and PriDe are not covered by fallback re-scoring (different failure semantics; see `fallback_analysis.py`).

Method	Model	Original	Fallback	Δ (pp)
Two-stage	GPT-4.1 mini	80.1	80.1	+0.0
Two-stage	Gemini 2.5 Flash	68.3	79.6	+11.3
Two-stage	Llama 3.1 8B (API)	64.9	64.9	+0.0
Two-stage	Llama 3.1 8B (local)	49.3	51.6	+2.3
Two-stage	Qwen 2.5 7B (API)	67.3	67.4	+0.1
Two-stage	Qwen 2.5 7B (local)	64.9	64.9	+0.0
Text extraction	GPT-4.1 mini	81.7	81.8	+0.1
Text extraction	Gemini 2.5 Flash	88.0	88.1	+0.1
Text extraction	Llama 3.1 8B (API)	62.3	62.6	+0.3
Text extraction	Llama 3.1 8B (local)	45.7	51.1	+5.4
Text extraction	Qwen 2.5 7B (API)	59.3	68.6	+9.3
Text extraction	Qwen 2.5 7B (local)	63.5	65.3	+1.8
Semantic matching	GPT-4.1 mini	45.8	57.0	+11.2
Semantic matching	Gemini 2.5 Flash	48.8	61.3	+12.5
Semantic matching	Llama 3.1 8B (API)	36.1	47.3	+11.2
Semantic matching	Llama 3.1 8B (local)	32.0	45.2	+13.2
Semantic matching	Qwen 2.5 7B (API)	38.8	47.7	+8.9
Semantic matching	Qwen 2.5 7B (local)	36.4	46.9	+10.5

Table 15: Fallback re-scoring on ARC-Challenge: original end-to-end accuracy vs. accuracy if unscorable outputs fall back to the baseline prediction, per model $\times$ method. Independent hypothesis and PriDe are not covered by fallback re-scoring (different failure semantics; see `fallback_analysis.py`).

Method	**Model**	**Original**	**Fallback**	**Δ (pp)**
Two-stage	GPT-4.1 mini	94.3	94.3	+0.0
Two-stage	Gemini 2.5 Flash	86.6	93.2	+6.6
Two-stage	Llama 3.1 8B (API)	79.1	79.1	+0.0
Two-stage	Llama 3.1 8B (local)	58.3	61.9	+3.6
Two-stage	Qwen 2.5 7B (API)	84.6	84.6	+0.0
Two-stage	Qwen 2.5 7B (local)	79.7	79.7	+0.0
Text extraction	GPT-4.1 mini	95.6	95.6	+0.0
Text extraction	Gemini 2.5 Flash	96.3	96.4	+0.1
Text extraction	Llama 3.1 8B (API)	82.7	82.7	+0.0
Text extraction	Llama 3.1 8B (local)	59.1	61.4	+2.3
Text extraction	Qwen 2.5 7B (API)	85.5	90.1	+4.6
Text extraction	Qwen 2.5 7B (local)	87.3	87.8	+0.5
Semantic matching	GPT-4.1 mini	52.1	62.6	+10.5
Semantic matching	Gemini 2.5 Flash	55.4	68.9	+13.5
Semantic matching	Llama 3.1 8B (API)	43.9	54.7	+10.8
Semantic matching	Llama 3.1 8B (local)	36.9	48.9	+12.0
Semantic matching	Qwen 2.5 7B (API)	47.8	57.6	+9.8
Semantic matching	Qwen 2.5 7B (local)	45.9	57.1	+11.2